

# Manipular la información de una tabla utilizando el lenguaje SQL para dar solución a un requerimiento

Este módulo se enfoca en el manejo de información en bases de datos PostgreSQL utilizando SQL, proporcionando las herramientas necesarias para crear, modificar y gestionar datos de manera eficiente y segura.

# El motor PostgreSQL y pgAdmin

PostgreSQL es un motor de base de datos relacional de código abierto, robusto y altamente extendido en la industria. Su potencia radica en:

## Cumplimiento del estándar SQL

Garantiza compatibilidad y portabilidad

## Soporte de transacciones ACID

Asegura integridad y consistencia de datos

## Extensibilidad

Tipos de datos, funciones y operadores definidos por el usuario

## Seguridad avanzada

Roles, privilegios y cifrado

## Integración

Compatible con Python, R y frameworks de análisis

En entornos profesionales se administra con pgAdmin, una interfaz gráfica que permite a los analistas y administradores conectarse al motor, ejecutar sentencias SQL, crear y modificar estructuras de datos, revisar registros y generar reportes sin necesidad exclusiva de línea de comandos.

# Conectarse a una base de datos PostgreSQL

Existen dos formas principales:

## 1. Consola psql (línea de comandos)

Ejemplo:

```
psql -h localhost -U usuario -d basedatos
```

- -h define el host (servidor)
- -U indica el usuario
- -d selecciona la base de datos inicial

## 2. pgAdmin (interfaz gráfica)

- Se instala junto con PostgreSQL
- Permite registrar servidores
- Solicita credenciales (usuario/contraseña)
- Presenta esquemas, tablas y funciones de manera visual



**Actividad sugerida:** conectarse en pgAdmin a una instancia local, navegar por las bases de datos y ubicar el esquema public, que es el predeterminado.

# Creación de una nueva base de datos PostgreSQL

Las bases de datos se crean con la instrucción:

```
CREATE DATABASE ventas2025;
```

En pgAdmin, el mismo proceso se logra con clic derecho en Databases → Create → Database.

Es importante definir:

## Propietario (OWNER)

Quién administra la base

## Codificación (ENCODING)

Por defecto UTF8, recomendable para soportar múltiples idiomas

## Tablespace

Directorio donde se guardan los archivos físicos

# Creación de permisos asociados a una base de datos

PostgreSQL gestiona permisos a través de roles (que pueden actuar como usuarios o grupos).

Ejemplo:

```
CREATE ROLE analista LOGIN PASSWORD 'seguro123';  
GRANT CONNECT ON DATABASE ventas2025 TO analista;
```

Esto crea un rol con permisos de conexión. Los permisos sobre objetos (tablas, vistas) se asignan luego con GRANT SELECT, GRANT INSERT, etc.

# Creación de tablas

Las tablas son el corazón de la base de datos. Se definen con CREATE TABLE, donde se especifican columnas, tipos de datos y restricciones.

Ejemplo de tabla de clientes:

```
CREATE TABLE clientes (  
  id_cliente SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  correo VARCHAR(150) UNIQUE,  
  fecha_registro DATE DEFAULT CURRENT_DATE,  
  activo BOOLEAN DEFAULT TRUE  
);
```

# Definición de columnas y tipos de datos

PostgreSQL soporta una amplia gama:

1

## Numéricos

INTEGER, BIGINT, NUMERIC, REAL



## Texto

CHAR, VARCHAR, TEXT



## Fechas y horas

DATE, TIMESTAMP, TIME



## Booleanos

BOOLEAN



## Especiales

UUID, JSONB, ARRAY

La elección correcta es fundamental para optimizar espacio y consultas.

# Restricciones y especificaciones en columnas

Las restricciones aseguran integridad de datos:



## PRIMARY KEY

Identificador único



## UNIQUE

Valores no repetidos



## NOT NULL

Evita valores nulos



## CHECK

Condiciones personalizadas



## FOREIGN KEY

Relación con otra tabla

Ejemplo:

```
CREATE TABLE pedidos (  
  id_pedido SERIAL PRIMARY KEY,  
  id_cliente INT NOT NULL REFERENCES clientes(id_cliente),  
  monto NUMERIC(10,2) CHECK (monto > 0)  
);
```



# Manipulación de los registros de una tabla

Una vez creadas las estructuras, el siguiente paso es manipular registros mediante DML (Data Manipulation Language).



# Inserción manual de valores en una tabla

Ejemplo:

```
INSERT INTO clientes (nombre, correo)
VALUES ('Ana Torres', 'ana.torres@mail.com');
```

Se pueden insertar múltiples registros:

```
INSERT INTO clientes (nombre, correo)
VALUES
('Carlos Pérez', 'carlosp@mail.com'),
('María Gómez', 'maria.g@mail.com');
```

 **Tip:** La inserción múltiple es más eficiente que múltiples INSERT individuales.

# Inserción de un CSV en una tabla

PostgreSQL permite cargar archivos CSV con el comando COPY:

```
COPY clientes(nombre, correo)
FROM '/ruta/clientes.csv'
DELIMITER ','
CSV HEADER;
```

En pgAdmin, esta función está disponible en Import/Export Data.

**Consideración:** validar que el CSV tenga los mismos tipos de datos que la tabla.

# Actualización y eliminación de registros

## Actualización de valores existentes

Se hace con UPDATE:

```
UPDATE clientes  
SET activo = FALSE  
WHERE id_cliente = 3;
```

Es recomendable incluir siempre un WHERE, para evitar actualizar todos los registros por error.

## Eliminación de registros específicos

La eliminación se realiza con DELETE:

```
DELETE FROM clientes  
WHERE id_cliente = 5;
```

# Consideraciones sobre el peligro de eliminar registros

## Eliminación sin WHERE

Vacía toda la tabla

## Proceso irreversible

A menos que exista un backup o se use una transacción con ROLLBACK

## Alternativa: Soft Delete

Usar un campo activo en lugar de borrado físico

Ejemplo de soft delete:

```
UPDATE clientes SET activo = FALSE WHERE id_cliente = 5;
```

# Parte Práctica – Ejemplos Integrados

A continuación, se propone un flujo práctico completo en PostgreSQL con pgAdmin.

01

## Crear la base de datos y roles

```
CREATE DATABASE tienda;  
CREATE ROLE gestor LOGIN PASSWORD  
'clave123';  
GRANT CONNECT ON DATABASE tienda  
TO gestor;
```

02

## Crear tablas

```
CREATE TABLE productos (  
  id_producto SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio NUMERIC(10,2) NOT NULL CHECK  
  (precio > 0),  
  stock INT DEFAULT 0  
);  
  
CREATE TABLE ventas (  
  id_venta SERIAL PRIMARY KEY,  
  id_producto INT REFERENCES  
  productos(id_producto),  
  cantidad INT CHECK (cantidad > 0),  
  fecha TIMESTAMP DEFAULT  
  CURRENT_TIMESTAMP  
);
```

03

## Insertar registros

```
INSERT INTO productos (nombre,  
precio, stock)  
VALUES ('Laptop', 750000, 10),  
('Mouse', 12000, 50);
```

04

## Insertar CSV (ejemplo práctico en pgAdmin)

- Usar menú Import Data
- Seleccionar archivo productos.csv
- Definir delimitador ,
- Ejecutar

05

## Actualizar registros

```
UPDATE productos  
SET stock = stock - 2  
WHERE id_producto = 1;
```

06

## Eliminar registros (con precaución)

```
DELETE FROM productos  
WHERE id_producto = 2;
```

07

## Consulta de validación

```
SELECT p.nombre, v.cantidad, v.fecha  
FROM ventas v  
JOIN productos p ON v.id_producto = p.id_producto;
```

Esto genera un reporte de ventas con nombres de productos, mostrando la integración entre tablas.

# Conclusión

Este aprendizaje permite:



**Crear estructuras de datos en PostgreSQL**



**Insertar, modificar y eliminar registros**



**Comprender riesgos y buenas prácticas en la manipulación de tablas**



**Aplicar operaciones SQL para dar solución a requerimientos reales en escenarios empresariales**

El dominio de estas técnicas es fundamental para cualquier profesional que trabaje con bases de datos en entornos empresariales modernos.

